EECE 7205: Computing Fundamentals for Engineers

Fall 2026

Lab Assignment #01 Binary Tree Interview Questions

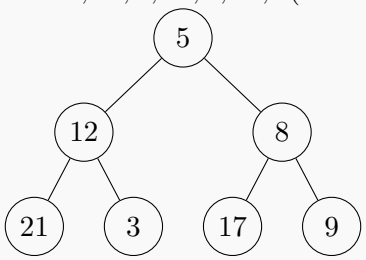
Topic: Binary Trees

Student Name	Eduardo Scopelli
NUID	003972121
Email	scopellirivas.e@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	10-11-2026

Checklist Item

Goal: Answer all 15 questions. Unless a question says otherwise, use the tree built from this array in level order (the node at index i has children at 2i + 1 and 2i + 2):

$$\text{arr} = 5, 12, 8, 21, 3, 17, 9 (n = 7)$$



For each question, submit the four parts below:

- **Pseudocode:** Write the algorithm as a procedure with numbered lines, in the same style as the lecture notes (Input, Output, Initial call, then the numbered lines).
- **Worked example:** Trace your pseudocode on the given input. For recursive solutions, list every call with the lines it executes and what it returns. For loops with a queue or stack, give one row per pass showing the queue or stack. State the final output and check it against the expected output.
- **Frequency count:** Give a table with one row per numbered line: Line, Operation, Cost, Frequency (as a formula in n, or h where needed) and Count for the example. Add the rows to get T(n), and check that your formula gives the same total as your example.
- **Final complexity:** Keep the dominant term of T(n) and state the best case, worst case and extra space using Theta notation. Explain in one or two sentences which input gives the worst case.

Counting rules. Every line costs 1. A test such as "if node = NULL" counts every time it is evaluated. A recursive call counts once on the line that makes it. Lines that only end a block (end if, end while, end procedure) are not counted. A tree with n nodes has n + 1 NULL child pointers.

Question 1: Maximum depth of a binary tree

Return the maximum depth of the tree: the number of edges on the longest path from the root to a leaf (an empty tree has depth -1).

- **Input:** Tree from arr
- **Expected output:** 2
- **Twist:** Would your frequency count change for a tree that is a single chain of 7 nodes?

Pseudocode

```
function maxDepth(node):
    if node = null {
        return -1
    }

    leftDepth ← maxDepth(node.left)
    rightDepth ← maxDepth(node.right)

    if leftDepth > rightDepth {
        return 1 + leftDepth
    } else {
        return 1 + rightDepth
    }
end function
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	if $node = null$	1	$2n + 1$	15
3	return -1	1	$n + 1$	8
6	$leftDepth \leftarrow maxDepth(node.left)$	1	n	7
7	$rightDepth \leftarrow maxDepth(node.right)$	1	n	7
9	if $leftDepth > rightDepth$	1	n	7
10	return $1 + leftDepth$	1	n	7
12	return $1 + rightDepth$	1	n	7

Final complexity

Time complexity (Best case): $T(n) = 7n + 2 \rightarrow O(n)$
Time complexity (Worst case): $T(n) = 7n + 2 \rightarrow O(n)$

Explanation: This algorithm will check every node of the tree recursively, starting from the root and moving down to both the left and right children of each subsequent node before comparing the accumulated height of each branch.

Twist: Would your frequency count change for a tree that is a single chain of 7 nodes?
Answer: As mentioned previously, the time complexity of this algorithm depends solely on the amount of nodes it will check, there is no distinction with the shape of the tree.

Worked example

Function Call	Lines executed	Cost
maxDepth(5) returns 2	if $node = null$	1
	$leftDepth \leftarrow maxDepth(node.left)$	1
	$rightDepth \leftarrow maxDepth(node.right)$	1
	if $leftDepth > rightDepth$	1
	return $1 + leftDepth$ or return $1 + rightDepth$	1
maxDepth(12) returns 1	if $node = null$	1
	$leftDepth \leftarrow maxDepth(node.left)$	1
	$rightDepth \leftarrow maxDepth(node.right)$	1
	if $leftDepth > rightDepth$	1
	return $1 + leftDepth$ or return $1 + rightDepth$	1
maxDepth(8) returns 1	if $node = null$	1
	$leftDepth \leftarrow maxDepth(node.left)$	1
	$rightDepth \leftarrow maxDepth(node.right)$	1
	if $leftDepth > rightDepth$	1
	return $1 + leftDepth$ or return $1 + rightDepth$	1
maxDepth(21) returns 0	if $node = null$	1
	$leftDepth \leftarrow maxDepth(node.left)$	1
	$rightDepth \leftarrow maxDepth(node.right)$	1
	if $leftDepth > rightDepth$	1
	return $1 + leftDepth$ or return $1 + rightDepth$	1
maxDepth(3) returns 0	if $node = null$	1
	$leftDepth \leftarrow maxDepth(node.left)$	1
	$rightDepth \leftarrow maxDepth(node.right)$	1
	if $leftDepth > rightDepth$	1
	return $1 + leftDepth$ or return $1 + rightDepth$	1
maxDepth(17) returns 0	if $node = null$	1
	$leftDepth \leftarrow maxDepth(node.left)$	1
	$rightDepth \leftarrow maxDepth(node.right)$	1
	if $leftDepth > rightDepth$	1
	return $1 + leftDepth$ or return $1 + rightDepth$	1
maxDepth(9) returns 0	if $node = null$	1
	$leftDepth \leftarrow maxDepth(node.left)$	1
	$rightDepth \leftarrow maxDepth(node.right)$	1
	if $leftDepth > rightDepth$	1
	return $1 + leftDepth$ or return $1 + rightDepth$	1
8*maxDepth(NULL) returns -1	if $node = null$	8
	return -1	8
Total:		51

$T(n = 7) = 7(7) + 2 = 51$✓

Question 2: Sum of all nodes

Return the sum of the values stored in all nodes of the tree.

- **Input:** Tree from arr
- **Expected output:** 75
- **Twist:** Write it once recursively and once with a queue (level order). Compare the two frequency counts and the extra space each needs.

(a) Recursive

Pseudocode

```
function sumTree(node):
    if node = null {
        return 0
    }

    leftSum ← sumTree(node.left)
    rightSum ← sumTree(node.right)

    return node.data + leftSum + rightSum
end function
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	if $node = null$	1	$2n + 1$	15
3	return 0	1	$n + 1$	8
6	$leftSum \leftarrow sumTree(node.left)$	1	n	7
7	$rightSum \leftarrow sumTree(node.right)$	1	n	7
9	return $node.data + leftSum + rightSum$	1	n	7

Final complexity

Time complexity (Best case): $T(n) = 6n + 2 \rightarrow O(n)$
Time complexity (Worst case): $T(n) = 6n + 2 \rightarrow O(n)$

Explanation: This algorithm will check every node of the tree recursively, starting from the root and moving down to both the left and right children of each subsequent node, returning the value of the node and both accumulated sums of the branches.

Space complexity (Best case): $S(n) = O(log(n))$
Space complexity (Worst case): $S(n) = O(n)$

Worked example

Function Call	Lines executed	Cost
sumTree(5) returns 75	if $node = null$	1
	$leftSum \leftarrow sumTree(node.left)$	1
	$rightSum \leftarrow sumTree(node.right)$	1
	return $node.data + leftSum + rightSum$	1
sumTree(12) returns 36	if $node = null$	1
	$leftSum \leftarrow sumTree(node.left)$	1
	$rightSum \leftarrow sumTree(node.right)$	1
	return $node.data + leftSum + rightSum$	1
sumTree(8) returns 34	if $node = null$	1
	$leftSum \leftarrow sumTree(node.left)$	1
	$rightSum \leftarrow sumTree(node.right)$	1
	return $node.data + leftSum + rightSum$	1
sumTree(21) returns 21	if $node = null$	1
	$leftSum \leftarrow sumTree(node.left)$	1
	$rightSum \leftarrow sumTree(node.right)$	1
	return $node.data + leftSum + rightSum$	1
sumTree(3) returns 3	if $node = null$	1
	$leftSum \leftarrow sumTree(node.left)$	1
	$rightSum \leftarrow sumTree(node.right)$	1
	return $node.data + leftSum + rightSum$	1
sumTree(17) returns 17	if $node = null$	1
	$leftSum \leftarrow sumTree(node.left)$	1
	$rightSum \leftarrow sumTree(node.right)$	1
	return $node.data + leftSum + rightSum$	1
sumTree(9) returns 9	if $node = null$	1
	$leftSum \leftarrow sumTree(node.left)$	1
	$rightSum \leftarrow sumTree(node.right)$	1
	return $node.data + leftSum + rightSum$	1
8*sumTree(NULL) returns 0	if $node = null$	8
	return -1	8
Total:		44

$T(n = 7) = 6(7) + 2 = 44$✓

(b) Queue

Pseudocode

```
function sumTree(node):
    queue q
    q.push(node)
    sum ← 0

    while not q.empty() {
        sum +=  q.front().data

        if q.front().left ≠ null {
```

```
10              q.push(q.front().left)
11          }
12          if q.front().right ≠ null {
13              q.push(q.front().right)
14          }
15          q.pop()
16      }
17      return sum
18  end function
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	*queue q*	1	1	1
3	$q.push(node)$	1	1	1
4	$sum \leftarrow 0$	1	1	1
6	while not $q.empty()$	1	$n+1$	8
7	$sum+ = q.front().data$	1	n	7
9	if $q.front().left \neq null$	1	n	7
10	$q.push(q.front().left)$	1	$\frac{n-1}{2}$	3
12	if $q.front().right \neq null$	1	n	7
13	$q.push(q.front().right)$	1	$\frac{n-1}{2}$	3
15	$q.pop()$	1	n	7
17	return *sum*	1	1	1

Final complexity

Time complexity (Best case): $T(n) = 6n + 4 \rightarrow O(n)$
Time complexity (Worst case): $T(n) = 6n + 4 \rightarrow O(n)$

Explanation: This algorithm will create a queue with the different nodes in the tree, where each iteration will check the first node in the queue, push any child nodes to the queue and add the current node's value before removing it from the queue. **Space complexity (Best case):** $S(n) = O(1)$
Space complexity (Worst case): $S(n) = O(n)$

Worked example

Iteration	Node	Queue	Lines executed	Cost
0	5	[5]	2-15	10
1	12	[12, 8]	6-15	7
2	8	[8, 21, 3]	6-15	7
3	21	[21, 3, 17, 9]	6-9, 12, 15	5
4	3	[3, 17, 9]	6-9, 12, 15	5
5	17	[17, 9]	6-9, 12, 15	5
6	9	[9]	6-9, 12, 15	5
7	NULL	[]	6, 17	2

$T(n = 7) = 6(7) + 4 = 46$✓

Question 3: Nodes with one child

Return the number of nodes that have a left child or a right child, but not both.

- **Input:** Tree from arr after INSERT(root, 4) (level order rules from the lecture).
- **Expected output:** 1 (node 21, whose only child is 4)
- **Twist:** Explain why a tree built from an array of n elements has exactly one such node when n is even and none when n is odd.

Pseudocode

```
function countSingleChild(node):
    if node = null {
        return 0
    }
    count ← 0
    if (node.left = null and node.right ≠ null) or (node.left ≠ null and node.right =
        null) {
        count ← 1
    }
    count +=  countSingleChild(node.left)
    count +=  countSingleChild(node.right)
    return count
end function
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	if $node = null$	1	$2n+1$	15
3	return 0	1	$n+1$	9
5	$count \leftarrow 0$	1	n	1
6	if ($node.left = null$ and $node.right \neq null$) or ($node.left \neq null$ and $node.right = null$)	4	n	8
7	$count \leftarrow 1$	1	k_n	1
9	$count+ = countSingleChild(node.left)$	1	n	8
10	$count+ = countSingleChild(node.right)$	1	n	8
11	return $count$	1	n	8

Final complexity

Time complexity (Best case): $T(n) = 11n + 2 + k_n = 11n + 2, (k_n = 0) \rightarrow O(n)$
Time complexity (Worst case): $T(n) = 11n + 2 + k_n = 12n + 1, (k_n = n - 1) \rightarrow O(n)$

Explanation: Similar to the first problem, this algorithm goes over every node of the tree up to the children pointers of the last nodes. In this case, with a conditional for any node with one child, this condition adds one additional operation for every node that has only one child, which ranges between 0 (all nodes have 0 or 2 children, usually meaning a complete binary tree) and $n - 1$

(all nodes are positioned sequentially, as a linked list).

Twist: Explain why a tree built from an array of n elements has exactly one such node when n is even and none when n is odd.
Answer: Assuming a binary tree is built following a level order, as we have a single unpaired node in the root for every other pair of nodes added, we ensure that each previous parent has both a left and right child, or has no children. When we add an additional unpaired node, we get a single parent node with an unmatched child (n is even without counting the root).

Worked example

Function Call	Lines executed	Cost
maxDepth(5) returns 1	if $node = null$	1
	$count \leftarrow 0$	1
	if ($node.left = null$ and $node.right \neq null$) or ($node.left \neq null$ and $node.right = null$)	4
	$count+ = countSingleChild(node.left)$	1
	$count+ = countSingleChild(node.right)$	1
	return $count$	1
maxDepth(12) returns 1	if $node = null$	1
	$count \leftarrow 0$	1
	if ($node.left = null$ and $node.right \neq null$) or ($node.left \neq null$ and $node.right = null$)	4
	$count+ = countSingleChild(node.left)$	1
	$count+ = countSingleChild(node.right)$	1
	return $count$	1
maxDepth(8) returns 0	if $node = null$	1
	$count \leftarrow 0$	1
	if ($node.left = null$ and $node.right \neq null$) or ($node.left \neq null$ and $node.right = null$)	4
	$count+ = countSingleChild(node.left)$	1
	$count+ = countSingleChild(node.right)$	1
	return $count$	1
maxDepth(21) returns 1	if $node = null$	1
	$count \leftarrow 0$	1
	if ($node.left = null$ and $node.right \neq null$) or ($node.left \neq null$ and $node.right = null$)	4
	$count \leftarrow 1$	1
	$count+ = countSingleChild(node.left)$	1
	$count+ = countSingleChild(node.right)$	1
	return $count$	1
maxDepth(3) returns 0	if $node = null$	1
	$count \leftarrow 0$	1
	if ($node.left = null$ and $node.right \neq null$) or ($node.left \neq null$ and $node.right = null$)	4
	$count+ = countSingleChild(node.left)$	1
	$count+ = countSingleChild(node.right)$	1
	return $count$	1

Function Call	Lines executed	Cost
maxDepth(17) returns 0	if $node = null$	1
	$count \leftarrow 0$	1
	if ($node.left = null$ and $node.right \neq null$) or ($node.left \neq null$ and $node.right = null$)	4
	$count+ = countSingleChild(node.left)$	1
	$count+ = countSingleChild(node.right)$	1
	return $count$	1
maxDepth(9) returns 0	if $node = null$	1
	$count \leftarrow 0$	1
	if ($node.left = null$ and $node.right \neq null$) or ($node.left \neq null$ and $node.right = null$)	4
	$count+ = countSingleChild(node.left)$	1
	$count+ = countSingleChild(node.right)$	1
	return $count$	1
maxDepth(4) returns 0	if $node = null$	1
	$count \leftarrow 0$	1
	if ($node.left = null$ and $node.right \neq null$) or ($node.left \neq null$ and $node.right = null$)	4
	$count+ = countSingleChild(node.left)$	1
	$count+ = countSingleChild(node.right)$	1
	return $count$	1
9*maxDepth(NULL) returns 0	if $node = null$	9
	return 0	9
Total:		91

$T(n = 7, k_n = 1) = 11(8) + 2 + (1) = 91$✓

Question 4: Are two trees identical?

Given the roots of two trees, return true if they have the same shape and the same value in every position, otherwise false.

- **Input:** Tree A from arr; tree B from arr; tree C from 5, 12, 8, 21, 3, 17, 10.
- **Expected output:** IDENTICAL(A, B) = true; IDENTICAL(A, C) = false
- **Twist:** For IDENTICAL(A, C), at which node does your algorithm first find a difference? What is the best case and the worst case for two trees of n nodes?

Pseudocode

```
function identical(A, B):
    queue qA
    queue qB
    qA.push(A)
    qB.push(B)

    while not qA.empty() and not qB.empty() {
        if qA.front() ≠ qB.front() {
            return false
        }
        if qA.front().left ≠ null and qB.front().left ≠ null {
            qA.push(qA.front().left)
            qB.push(qB.front().left)
        }
        if qA.front().right ≠ null and qB.front().right ≠ null {
            qA.push(qA.front().right)
            qB.push(qB.front().right)
        }
        qA.pop()
        qB.pop()
    }
    return qA.size() = qB.size()
end function
```

Frequency count

Considering $N = min(n_a, n_b)$

Line	Operation	Cost	Frequency	Count for example
2	$queue\ qA$	1	1	1
3	$queue\ qB$	1	1	1
4	$qA.push(A)$	1	1	1
5	$qB.push(B)$	1	1	1
7	while not $qA.empty()$ and not $qB.empty()$	2	$N+1$	16
8	if $qA.front() \neq qB.front()$	1	N	7
9	return $false$	1	1	0, 1
11	if $qA.front().left \neq null$ and $qB.front().left \neq null$	2	N	12, 14
12	$qA.push(qA.front().left)$	1	$\frac{N-1}{2}$	2, 3
13	$qB.push(qB.front().left)$	1	$\frac{N-1}{2}$	2, 3
15	if $qA.front().right \neq null$ and $qB.front().right \neq null$	2	N	12, 14
16	$qA.push(qA.front().right)$	1	$\frac{N-1}{2}$	2, 3
17	$qB.push(qB.front().right)$	1	$\frac{N-1}{2}$	2, 3
19	$qA.pop()$	1	N	6, 7
20	$qB.pop()$	1	N	6, 7
22	return $qA.size() = qB.size()$	2	1	2, 0

Final complexity

Time complexity (Best case): $T(n) = 8 \rightarrow O(1)$
Time complexity (Worst case): $T(n) = 11N + 5, N = min(n_a, n_b) \rightarrow O(N)$

Explanation: For this algorithm, if the first node of the two trees is different, it immediately returns $false$ after initializing the starter variables. In the case that it does have coinciding node values at the beginning of each tree, through two queue variables we iterate over each pair of nodes to check if they are the same. Once one of the queues is empty, we exit the loop and do one final check between both queues to make sure neither tree has any additional nodes.

Twist: For IDENTICAL(A, C), at which node does your algorithm first find a difference? What is the best case and the worst case for two trees of n nodes?
Answer: Between trees A and C, the last node is the only one that's different, which means that this algorithm will get to the 7_{th} iteration and immediately return $false$ in the first two lines.

Worked example

Iteration	Nodes	Queue A	Queue B	Lines executed	Cost
0	$5 = 5$	[5]	[5]	2-8, 11-20	17
1	$12 = 12$	$[12, 8]$	$[12, 8]$	7-8, 11-20	13
2	$8 = 8$	$[8, 21, 3]$	$[8, 21, 3]$	7-8, 11-20	13
3	$21 = 21$	$[21, 3, 17, 9]$	$[21, 3, 17, 9]$	7-8, 11, 15, 19-20	9
4	$3 = 3$	$[3, 17, 9]$	$[3, 17, 9]$	7-8, 11, 15, 19-20	9
5	$17 = 17$	$[17, 9]$	$[17, 9]$	7-8, 11, 15, 19-20	9
6	$9 = 9$ (C: $9 \neq 10$)	[9]	[9]	7-8, 11, 15, 19-20 (C: 7-9)	9 (C: 4)
7	$0 = 0$	[]	[]	7, 22	3

$T(n = 7) = 11(7) + 5 = 82$✓

Question 5: Minimum value in a binary tree

Return the smallest value in the tree. The tree is not a binary search tree, so every node must be checked.

- **Input:** Tree from arr.
- **Expected output:** 3
- **Twist:** How would the algorithm and its complexity change if the tree were a binary search tree?

Pseudocode

```
function minValue(node):
    queue q
    q.push(node)
    min ← q.front().data

    while not q.empty() {
        if q.front().data < min {
            min ← q.front()
        }
        if q.front().left ≠ null {
            q.push(q.front().left)
        }
        if q.front().right ≠ null {
            q.push(q.front().right)
        }
        q.pop()
    }
    return min
end function
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	$queue\ q$	1	1	1
3	$q.push(node)$	1	1	1
4	$min \leftarrow q.front().data$	1	1	1
6	while not $q.empty()$	1	$n+1$	8
7	if $q.front().data < min$	1	n	7
8	$min \leftarrow q.front().data$	1	n	7
10	if $q.front().left \neq null$	1	n	7
11	$q.push(q.front().left)$	1	$\frac{n-1}{2}$	3
13	if $q.front().right \neq null$	1	n	7
14	$q.push(q.front().right)$	1	$\frac{n-1}{2}$	3
16	$q.pop()$	1	n	7
18	return min	1	1	1

Final complexity

Time complexity (Best case): $T(n) = 6n + 4 \rightarrow O(n)$
Time complexity (Worst case): $T(n) = 7n + 4 \rightarrow O(n)$

Explanation: For this algorithm, we use a queue to go through every node, comparing each one with our initial minimum value and overwriting whenever a smaller value is found. Between the best and worst case, it depends on how many times the min value is overwritten (0 times if the first value is the smallest, n times if it's the last value).

Twist: How would the algorithm and its complexity change if the tree were a binary search tree?
Answer: If the tree were a binary search tree in which every parent has a smaller value as it's left child, the smallest value of the tree is found in the left-most node, which means our time complexity comes down to $O(log_2 n)$.

Worked example

Iteration	Node	Min	Queue	Lines executed	Cost
0	5	5	[5]	2-15	10
1	12	5	[12, 8]	6-15	7
2	8	5	[8, 21, 3]	6-15	7
3	21	5	[21, 3, 17, 9]	6-9, 12, 15	5
4	3	3	[3, 17, 9]	6-9, 12, 15	6
5	17	3	[17, 9]	6-9, 12, 15	5
6	9	3	[9]	6-9, 12, 15	5
7	NULL	3	[]	6, 17	2

$T(n = 7) = 6(7) + 4 + (1)$ **(1 min swap)** $= 47$✓

Question 6: Invert / Mirror binary tree

Swap the left and right child of every node, so the tree becomes its mirror image. Return the root.

- **Input:** Tree from arr.
- **Expected output:** level order after inverting: 5, 8, 12, 9, 17, 3, 21
- **Twist:** Give the inorder traversal before (21 12 3 5 17 8 9) and after inverting. What do you notice, and why does it happen?

Pseudocode

```
function minValue(node):
    queue q
    q.push(node)

    while not q.empty() {
        swap(q.front().left, q.front().right)

        if q.front().left ≠ null {
            q.push(q.front().left)
        }
        if q.front().right ≠ null {
            q.push(q.front().right)
        }
        q.pop()
    }
    return node
end function
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	*queue q*	1	1	1
3	$q.push(node)$	1	1	1
5	while not $q.empty()$	1	$n+1$	8
6	$swap(q.front().left, q.front().right)$	1	n	7
8	if $q.front().left \neq null$	1	n	7
9	$q.push(q.front().left)$	1	$\frac{n-1}{2}$	3
11	if $q.front().right \neq null$	1	n	7
12	$q.push(q.front().right)$	1	$\frac{n-1}{2}$	3
14	$q.pop()$	1	n	7
16	return *node*	1	1	1

Final complexity

Time complexity (Best case): $T(n) = 6n + 3 \rightarrow O(n)$
Time complexity (Worst case): $T(n) = 6n + 3 \rightarrow O(n)$

Explanation: This algorithm uses a queue to go through every node in a tree and swap it's

left and right pointers before moving on to the next. In the cases of leaves or nodes with only one child, the same swap occurs, but with null values whenever a child doesn't exist.

Twist: Give the inorder traversal before (21 12 3 5 17 8 9) and after inverting. What do you notice, and why does it happen?
Answer: After inverting we get a inorder traversal of 9, 8, 17, 5, 3, 12, 21, which is the direct inverse of the original inorder traversal. This is due to the inorder's logic of reading nodes by the rule (left, node, right) and this algorithm swapping every left and right node, which when done for every node in the tree turns every interior node to have the structure (right, node, left).

Worked example

Iteration	Node	Queue	Reordered children	Lines executed	Cost
0	5	[5]	[8, 12]	2-14	9
1	8	[8, 12]	[9, 17]	5-14	7
2	12	[12, 9, 17]	[3, 21]	5-14	7
3	9	[9, 17, 13, 21]	[null, null]	5-8, 11, 14	5
4	17	[17, 3, 21]	[null, null]	5-8, 11, 14	5
5	3	[3, 21]	[null, null]	5-8, 11, 14	5
6	21	[21]	[null, null]	5-8, 11, 14	5
7	NULL	[]	[]	5, 16	2

$T(n = 7) = 6(7) + 3 = 45$✓

Question 7: Print each level on its own line, and the right side view

Print the nodes of each level on a separate line. Then return the right side view: the last node of each level, which is what you see when looking at the tree from the right

- **Input:** Tree from arr.
- **Expected output:** levels: [5], [12, 8], [21, 3, 17, 9]; right side view: 5, 8, 9
- **Twist:** Your frequency count will depend on both n and the number of levels h + 1. Show where each term comes from.

Pseudocode

```
function printTree(node):
    queue q
    q.push(node)
    rightSide ← []

    while not q.empty() {
        width ← q.size()
        for (i ← 0; i < width; i++) {
            temp ← q.front()
            q.pop()

            cout << temp.data
            if i = width - 1 {
                rightSide.append(temp)
            }
            if q.front().left ≠ null {
                q.push(q.front().left)
            }
            if q.front().right ≠ null {
                q.push(q.front().right)
            }
        }
        cout << endl
    }
    return rightSide
end function
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	$queue\ q$	1	1	1
3	$q.push(node)$	1	1	1
4	$rightSide \leftarrow []$	1	1	1
6	while not $q.empty()$	1	$h+2$	8
7	$width \leftarrow q.size()$	1	$h+1$	1
8	for $(i \leftarrow 0; i < width; i++)$	3	$2h+2n+1$	20
9	$temp \leftarrow q.front()$	1	n	7
10	$q.pop()$	1	n	7
12	$cout << temp.data$	1	n	7
13	if $i = width - 1$	1	n	7
14	$rightSide.append(temp)$	1	$h+1$	3
16	if $q.front().left \neq null$	1	n	7
17	$q.push(q.front().left)$	1	$\frac{n-1}{2}$	3
19	if $q.front().right \neq null$	1	n	7
20	$q.push(q.front().right)$	1	$\frac{n-1}{2}$	3
23	$cout << endl$	1	$h+1$	7
16	return $rightSide$	1	1	1

Final complexity

Time complexity (Best case): $T(n) = 9n + 6h + 9 \rightarrow O(n, h)$
Time complexity (Worst case): $T(n) = 15n + 3(h = n - 1) \rightarrow O(n)$

Explanation: For this algorithm the main while loop loops through the levels of the tree, causing the time complexity to depend on the value of the $height + 1$, while the for loop loops through the nodes, which depends on n. Additionally, within this for loop there's an additional if statement to check if a node is the right most node in the tree and append these nodes to an array to be returned by the function.

Twist: Your frequency count will depend on both n and the number of levels h + 1. Show where each term comes from.
Answer: As mentioned before, the algorithm uses a while loop to iterate over each level of the binary tree $(h + 1)$ and an internal for loop to iterate over the nodes in each level (n).

Worked example

# (h)	# (n)	Node	Queue	Printed	Lines executed	Cost
0	0	5	[5]	5	2-23	19
1	1	12	[12, 8]	12	6-13, 16-20	13
	2	8	[8, 21, 3]	12 8	8-23	13
2	3	21	[21, 3, 17, 9]	21	6-13, 16, 19	11
	4	3	[3, 17, 9]	21 3	8-13, 16, 19	8
	5	17	[17, 9]	21 3 17	8-13, 16, 19	8
	6	9	[9]	21 3 17 9	8-16, 19, 23-25	12

$T(n = 7) = 9(7) + 6(2) + 9 = 84$✓

Question 8: Zigzag / Spiral level order traversal

Visit level 0 from left to right, level 1 from right to left, level 2 from left to right, and so on.

- **Input:** Tree from arr.
- **Expected output:** 5, 8, 12, 21, 3, 17, 9
- **Twist:** Solve it with two stacks, or with a queue plus reversing every second level. Which uses less extra work in your frequency count?

Pseudocode

```
function printTree(node):
    stack s1
    stack s2
    s1.push(node)

    while not s1.empty() or s2.empty() {
        while not s1.empty() {
            temp ← s1.top()
            s1.pop()
            cout << temp.data

            if temp.left ≠ null {
                s2.push(temp.left)
            }
            if temp.right ≠ null {
                s2.push(temp.right)
            }
        }
        while not s2.empty() {
            temp ← s2.top()
            s2.pop()
            cout << temp.data

            if temp.right ≠ null {
                s1.push(temp.right)
            }
            if temp.left ≠ null {
                s1.push(temp.left)
            }
        }
    }
end function
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	$stack\ s1$	1	1	1
3	$stack\ s2$	1	1	1
4	$s1.push(node)$	1	1	1
6	while not $s1.empty()$ or not $s2.empty()$	1	$\lceil \frac{h+1}{2} \rceil + 1$	3
7	while not $s1.empty()$	1	$n_{evenL} + \lceil \frac{h+1}{2} \rceil$	7
8	$temp \leftarrow s1.top()$	1	n_{evenL}	5
9	$s1.pop()$	1	n_{evenL}	5
10	$cout << temp.data$	1	n_{evenL}	5
12	if $temp.left \neq null$	1	n_{evenL}	5
13	$s2.push(temp.left)$	1	n_{evenL_ch}	1
15	if $temp.right \neq null$	1	n_{evenL}	5
16	$s2.push(temp.right)$	1	n_{evenL_ch}	1
19	while not $s2.empty()$	1	$n_{oddL} + \lfloor \frac{h+1}{2} \rfloor$	3
20	$temp \leftarrow s2.top()$	1	n_{oddL}	2
21	$s2.pop()$	1	n_{oddL}	2
22	$cout << temp.data$	1	n_{oddL}	2
24	if $temp.right \neq null$	1	n_{oddL}	2
25	$s1.push(temp.right)$	1	n_{oddL_ch}	2
27	if $temp.left \neq null$	1	n_{oddL}	2
28	$s1.push(temp.left)$	1	n_{oddL_ch}	2

Final complexity

Time complexity:
$m = \lceil \frac{h+1}{2} \rceil; n = n_{evenL} + n_{oddL}$
$T(n) = 6n + 2n_{evenL_ch} + 2n_{oddL_ch} + m + h + 5 \rightarrow O(n, h)$

Explanation: For this algorithm we use two stacks as queues for both left-to-right runs as well as right-to-left runs. By using stacks with the Last-In, First-Out principle, we are able to reverse the order of a typical level order traversal every other level of a binary tree. As for the time complexity, this does depend on a few additional special cases or variables: The number of nodes on even and odd levels (which can be added up to n), the amount of nodes in these two subcategories with children, and the number of levels in the binary tree.

Twist: Solve it with two stacks, or with a queue plus reversing every second level. Which uses less extra work in your frequency count?
Answer: By using two stacks, we can directly create a queue for every pass of the binary tree no matter the direction, as if we were using a regular queue. If we were to use a queue and reversing the order every level, this would involve additional operations to complete said inversion of the queue every two levels on top of the first traversal (the time complexity would stay at $O(n)$, but the solution involving reversing the queue would have a larger coefficient for n).

Worked example

Outer Loop	Direction	S1	S2	Node	Printed	Lines executed	Cost
0	$\rightarrow$	[5]	[]	5	5	2-16	13
	$\leftarrow$	[]	[12, 8]	8	8	19-28	8
		[9, 17]	[12]	12	8, 12	19-28	9
1	$\rightarrow$	[9, 17, 3, 21]	[]	21	21, 3, 17, 9	6-12, 15	7
		[9, 17, 3]	[]	3	21, 3	7-12, 15	6
		[9, 17]	[]	17	21, 3, 17	7-12, 15	6
		[9]	[]	9	21, 3, 17, 9	7-12, 15	8

$T(n) = 6(7) + 2(1) + 2(2) + (2) + (2) + 5 = 57$✓

Question 9: Is the tree complete?

A tree is complete if every level is full except possibly the last, and the last level is filled from the left. Return true or false.

- **Input:** tree P from arr; tree Q given in level order with a gap: 5, 12, 8, NULL, 3, 17, 9.
- **Expected output:** complete(P) = $true$; complete(Q) = $false$
- **Twist:** Explain why INSERT and DELETE from the lecture always keep a tree complete.

Pseudocode

```
function complete(node):
    queue q
    q.push(node)
    prevNull ← false

    while not q.empty() {
        temp ← q.front()
        q.pop()
        if temp = null {
            prevNull ← true
        } else {
            if prevNull {
                return false
            }
            q.push(temp.left)
            q.push(temp.right)
        }
    }
    return true
end function
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	$queue\ q$	1	1	1
3	$q.push(node)$	1	1	1
4	$prevNull \leftarrow false$	1	1	1
6	while not $q.empty()$	1	$2n+2$	16
7	$temp \leftarrow q.front()$	1	$2n+1$	15
8	$q.pop()$	1	$2n+1$	15
9	if $temp = null$	1	$2n+1$	15
10	$prevNull \leftarrow true$	1	$n+1$	8
12	if $prevNull$	1	n	7
13	$return\ false$	1	0	0, 1
15	$q.push(temp.left)$	1	n	7
16	$q.push(temp.right)$	1	n	7
19	$return\ true$	1	1	1, 0

Final complexity

Time complexity (Best case): $T(n_{m_index}) = 7n_{m_index} + 7 \rightarrow O(n_{m_index})$
Time complexity (Worst case): $T(n) = 12n + 10 \rightarrow O(n)$

Explanation: This algorithm uses a queue to run a level order traversal across the tree, checking every node (including the null children of the leaves) and keeping track if a missing node is identified (n_m). If a missing node is identified, the loop continues with the next nodes, in which if any of them does have a value it immediately returns *true* as it is not a complete tree. As the time complexity depends on whether there is a missing node, the best case in terms of time (besides an empty tree) would occur whenever there is a missing node, as this will cut the iterations of the loop at the index of said missing node. In a special case, if $n > 2$ and the second node is missing (first left child, $n_{m_index} = 2$), the time complexity will always be 21 as the loop will always exit on the node following the missing one at index 1, no matter the amount of nodes following the missing one.

Twist: Solve it with two stacks, or with a queue plus reversing every second level. Which uses less extra work in your frequency count?
Answer: By using two stacks, we can directly create a queue for every pass of the binary tree no matter the direction, as if we were using a regular queue. If we were to use a queue and reversing the order every level, this would involve additional operations to complete said inversion of the queue every two levels on top of the first traversal (the time complexity would stay at $O(n)$, but the solution involving reversing the queue would have a larger coefficient for n).

Worked example

Tree P:

Iteration	Node	Queue	Lines executed	Cost
0	5	$[5]$	2-15	10
1	12	$[12, 8]$	6-15	7
2	8	$[8, 21, 3]$	6-15	7
3	21	$[21, 3, 17, 9]$	6-9, 12, 15	7
4	3	$[3, 17, 9, null]$	6-9, 12, 15	7
5	17	$[17, 9, null, null]$	6-9, 12, 15	7
6	9	$[9, null, null, null]$	6-9, 12, 15	7
7-10	NULL	$[null, null, null, null]$	6, 17	$8(5) + 2 = 42$

$T(n) = 12(7) + 10 = 94$✓

Tree Q:

Iteration	Node	Queue	Lines executed	Cost
0	5	$[5]$	2-15	10
1	12	$[12, 8]$	6-15	7
2	8	$[8, null, 3]$	6-15	7
3	NULL	$[null, 3, 17, 9]$	6-10	5
4	3	$[3, 17, 9, null]$	6-9, 12-13	6

$T(n) = 7(4) + 7 = 35$✓

Question 10: Lowest common ancestor

Given two values a and b in the tree, return their lowest common ancestor: the deepest node that has both a and b in its subtree (a node counts as being in its own subtree).

- **Input:** The tree from arr, a, b.
- **Expected output:** LCA(21, 3) = 12; LCA(21, 9) = 5; LCA(17, 8) = 8
- **Twist:** Solve it in one of two ways: (1) find both root-to-node paths with FINDPATH and compare them, or (2) a single recursive pass. Compare the frequency counts of the two methods.

Pseudocode

```
function LCA(node, a, b):
    pathA ← []
    pathB ← []

    findPath(node, a, pathA)
    findPath(node, b, pathB)

    i ← 0
    while i < pathA.size() and i < pathB.size() and pathA[i] = pathB[i] {
        i++
    }
    return pathA[i - 1]
end function
function findPath(node, target, path):
    if node = null {
        return false
    }
    path.push(node)
    if node.data = target {
        return true
    }
    if findPath(node.left, target, path) {
        return true
    }
    if findPath(node.right, target, path) {
        return true
    }
    path.pop()
    return false
end function
```

Frequency count

$s = min(pathA.size(), pathB.size())$;
$p = p_a + p_b$: Position of a and b in Pre-order; $n_{null} = n_{nullA} + n_{nullB}$: Null children before a and b;
$d_L = d_{aL} + d_{bL}$: Depth of a and b on left paths; $d_R = d_{aR} + d_{bR}$: Depth of a and b on right paths;
$e_{a/bL}, e_{a/bR}$: Left and right edges to a and b in Pre-order

Line	Operation	Cost	Frequency	Count for example
2	$pathA \leftarrow []$	1	1	1
3	$pathB \leftarrow []$	1	1	1
5	$findPath(node, a, pathA)$	1	1	1
6	$findPath(node, b, pathB)$	1	1	1
8	$i \leftarrow 0$	1	1	1
9	$while\ i < pathA.size()$ and $i < pathB.size()$ and $pathA[i] = pathB[i]$	3	$s + 1$	3
10	$i++$	1	s	2
12	$return\ pathA[i]$	1	1	1
15	if $node = null$	1	$p + n_{null}$	9
16	$return\ false$	1	n_{null}	2
18	$path.push(node)$	1	p	7
19	if $node.data = target$	1	p	7
20	$return\ true$	1	2	2
22	if $findPath(node.left, target, path)$	1	$e_{aL} + e_{bL}$	5
23	$return\ true$	1	d_L	3
25	if $findPath(node.right, target, path)$	1	$e_{aR} + e_{bR}$	2
26	$return\ true$	1	d_R	1
28	$path.pop()$	1	$n - h - 1$ for right branch	0
29	$return\ false$	1	$n - h - 1$ for right branch	0

Final complexity

Time complexity (Best case): $T(n) = 19 \rightarrow O(n_{m_index})$

Explanation: This algorithm recursively creates the path for each of the values of a and b, starting from the root and going down the branches until the specific node value is found. This is completed using a pre-order traversal, starting from a node and going down it's left and right branches, in which for each iteration we check if the next node is either null or the target. Once both paths for a and b are found, we run one last loop that checks every element of both paths, in which when an element that's different is found or we've iterated past the size of the path arrays, we return the previous node in either of the paths, which should be the last coinciding node for both. For the time complexity, this algorithm depends on the positions of a and b within the tree, complicating the calculation of a fixed time complexity function based on the height or the number of elements in the tree.

As for best and worst cases, the best case would be if both a and b are the root of the tree, for which both recursive functions only go down one level before returning the complete paths, with the final loop only iterating over the first and only value in both paths to return the lowest common

ancestor. As for the worst case, this one occurs when both a and b are the last nodes in a complete binary tree, which causes the algorithm to go down every previous branch recursively (up to $2n - 1$ times for the n nodes and $n - 1$ previous null children) before finding the final paths for a and b. Finally, once both paths are found the last outer loop still has to iterate over these elements $h + 2$ times before finding this lowest common ancestor.

Twist: Solve it in one of two ways: (1) find both root-to-node paths with FINDPATH and compare them, or (2) a single recursive pass. Compare the frequency counts of the two methods.
Answer: The algorithm shown above uses a FINDPATH helper function to recursively go down each path of the binary tree before finding the target value, which causes the time complexity to mostly depend on the position of the target in pre-order. If instead a single recursive pass is utilized to directly find the lowest common ancestor, we essentially half the amount of recursions being processed, as with a single one we are looking for both target values, rather than separately building path arrays for targets a and b. In terms of the time complexity, while both algorithms could still be considered as $O(n)$ (directly dependent on the amount of nodes), method 2 would have a smaller coefficient for n compared to method 1, meaning a slight and linear reduction of operations being required.

Worked example

Function Call	Path	Lines executed	Cost
findPath(5,$a = 21$,pathA) returns true	[5]	if $node = null$	1
		$path.push(node)$	1
		if $node.data = target$	1
		if $findPath(12, a, pathA)$	1
		$return\ true$	1
findPath(12,$a = 21$,pathA) returns true	[5, 12]	if $node = null$	1
		$path.push(node)$	1
		if $node.data = target$	1
		if $findPath(21, a, pathA)$	1
		$return\ true$	1
findPath(21,$a = 21$,pathA) returns true	[5, 12, 21]	if $node = null$	1
		$path.push(node)$	1
		if $node.data = target$	1
		$return\ true$	1
findPath(5,$b = 3$,pathB) returns true	[5]	if $node = null$	1
		$path.push(node)$	1
		if $node.data = target$	1
		if $findPath(12, b, pathB)$	1
		$return\ true$	1
findPath(12,$b = 3$,pathB) returns true	[5, 12]	if $node = null$	1
		$path.push(node)$	1
		if $node.data = target$	1
		if $findPath(21, b, pathB)$	1
		if $findPath(3, b, pathB)$	1
		$return\ true$	1
findPath(21,$b = 3$,pathB) returns false	[5, 12]	if $node = null$	1
		$path.push(node)$	1
		if $node.data = target$	1
		if $findPath(null, b, pathB)$	1
		if $findPath(null, b, pathB)$	1
		$path.pop()$	1
		$return\ false$	1
findPath(3,$b = 3$,pathA) returns true	[5, 12, 3]	if $node = null$	1
		$path.push(node)$	1
		if $node.data = target$	1
		$return\ true$	1

Question 11: Build a tree from its pre-order and in-order traversals

Given the preorder and inorder traversals of a tree with distinct values, rebuild the tree and return its root. Hint: the first preorder value is the root; its position in the inorder list splits the left and right subtrees.

- **Input:** Pre-order = 5, 12, 21, 3, 8, 17, 9; In-order = 21, 12, 3, 5, 17, 8, 9
- **Expected output:** the tree from arr; check: level order 5, 12, 8, 21, 3, 17, 9 and postorder 21, 3, 12, 17, 9, 8, 5
- **Twist:** If you search the inorder list for the root each time, what is the worst-case frequency count? Show how a lookup table (value to inorder index) reduces it to Theta(n).

Pseudocode

```
Class Solution:
    inorderMap[int: int]
    preorderI ← 0
    function buildTree(preorder, inorder):
        n ← inorder.size()
        inorderMap.clear()
        preorderI ← 0

        for i ← 0; i < n; i++ {
            inorderMap[inorder[i]] ← i
        }
        root ← helper(preorder, 0, n - 1)
        return root
    end function
    function helper(preorder, left, right):
        if left > right {
            return null
        }
        rootVal ← preorder[preorderI]
        preorderI++
        new node(inorderMap[rootVal])

        inorderI ← inorderMap[rootVal]
        node.left ← helper(preorder, left, inorderI - 1)
        node.right ← helper(preorder, inorderI + 1, right)

        return node
    end function
end Class
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	$inorderMap[int, int]$	1	1	1
3	$preorderI$	1	1	1
4	$buildTree(preorder, inorder)$	1	1	1
5	$n \leftarrow inorder.size()$	1	1	1
6	$inorderMap.clear()$	1	1	1
7	$preorderI \leftarrow 0$	1	1	1
9	for $i \leftarrow 0; i < n; i++$	3	$2n + 2$	16
10	$inorderMap[inorder[i]] \leftarrow i$	1	n	7
12	$root \leftarrow helper(preorder, 0, n - 1)$	1	1	1
13	$return\ root$	1	1	1
16	if $left > right$	1	$2n + 1$	15
17	$return\ null$	1	$n + 1$	8
19	$rootVal \leftarrow preorder[preorderI]$	1	n	7
20	$preorderI++$	1	n	7
21	new $node(inorderMap[rootVal])$	1	n	7
23	$inorderI \leftarrow inorderMap[rootVal]$	1	n	7
24	$node.left \leftarrow helper(preorder, left, inorderI - 1)$	1	n	7
25	$node.right \leftarrow helper(preorder, inorderI + 1, right)$	1	n	7
27	$return\ node$	1	n	7

Final complexity

Time complexity: $T(n) = 13n + 12 \rightarrow O(n)$

Explanation: This algorithm recursively splits the inorder input to split the list into the various subtrees, and the preorder input to determine the root of each subtree. As this algorithm runs for every element in the preorder list to create each node, it's complexity is determined by the amount of nodes n. Additionally, at the beginning a map variable is initialized an created with the node value: index pairs to simplify the identification of these roots within the inorder list.

As for best and worst cases, this algorithm depends completely on the amount of nodes to add to the tree, so there is no real variation between a "best" and "worst" case unless the amount of nodes n is changed.

Twist: If you search the inorder list for the root each time, what is the worst-case frequency count?
Answer: If instead of a map we look for the index of the node within the inorder list for every iteration, this can increase our complexity by up to an order of n. In the best case, the index we're looking for is always at the beginning of the list, for which the time complexity wouldn't really change, which for the tree means it is skewed to the right. In the case where the search always returns the index in the middle of the inorder, essentially splitting the list in half with every iteration, this results in an increase in the time complexity of $logn$. If instead the search always returns the index at the end of the inorder list, this increases the time complexity by an order of n up to $O(n^2)$, giving us our worst case.

Worked example

Function Call	Lines executed	Cost
buildTree(preorder,inorder) returns the root	$inorderMap[int : int]$	1
	$preorderI \leftarrow 0$	1
	$buildTree(preorder, inorder)$	1
	$n \leftarrow inorder.size()$	1
	$inorderMap.clear()$	1
	if $preorderI \leftarrow 0$	1
	for $i \leftarrow 0; i < n; i++$	16
	$inorderMap[inorder[i]] \leftarrow i$	7
	$root \leftarrow helper(preorder, 0, n-1)$	1
	$return\ root$	1
helper(preorder,left,right) returns the node 5	if $left > right$	1
	$rootVal \leftarrow preorder[preorderI]$	1
	$preorderI++$	1
	new $node(inorderMap[rootVal])$	1
	$inorderI \leftarrow inorderMap[rootVal]$	1
	$node.left \leftarrow helper(preorder, left, inorderI-1)$	1
	$node.right \leftarrow helper(preorder, inorderI+1, right)$	1
	$return\ node$	1
helper(preorder,left,right) returns the node 12	if $left > right$	1
	$rootVal \leftarrow preorder[preorderI]$	1
	$preorderI++$	1
	new $node(inorderMap[rootVal])$	1
	$inorderI \leftarrow inorderMap[rootVal]$	1
	$node.left \leftarrow helper(preorder, left, inorderI-1)$	1
	$node.right \leftarrow helper(preorder, inorderI+1, right)$	1
	$return\ node$	1
helper(preorder,left,right) returns the node 21	if $left > right$	1
	$rootVal \leftarrow preorder[preorderI]$	1
	$preorderI++$	1
	new $node(inorderMap[rootVal])$	1
	$inorderI \leftarrow inorderMap[rootVal]$	1
	$node.left \leftarrow helper(preorder, left, inorderI-1)$	1
	$node.right \leftarrow helper(preorder, inorderI+1, right)$	1
	$return\ node$	1
helper(preorder,left,right) returns the node 3	if $left > right$	1
	$rootVal \leftarrow preorder[preorderI]$	1
	$preorderI++$	1
	new $node(inorderMap[rootVal])$	1
	$inorderI \leftarrow inorderMap[rootVal]$	1
	$node.left \leftarrow helper(preorder, left, inorderI-1)$	1
	$node.right \leftarrow helper(preorder, inorderI+1, right)$	1
	$return\ node$	1

Function Call	Lines executed	Cost
helper(preorder,left,right) returns the node 8	if $left > right$	1
	$rootVal \leftarrow preorder[preorderI]$	1
	$preorderI++$	1
	new $node(inorderMap[rootVal])$	1
	$inorderI \leftarrow inorderMap[rootVal]$	1
	$node.left \leftarrow helper(preorder, left, inorderI - 1)$	1
	$node.right \leftarrow helper(preorder, inorderI + 1, right)$	1
	$return\ node$	1
helper(preorder,left,right) returns the node 17	if $left > right$	1
	$rootVal \leftarrow preorder[preorderI]$	1
	$preorderI++$	1
	new $node(inorderMap[rootVal])$	1
	$inorderI \leftarrow inorderMap[rootVal]$	1
	$node.left \leftarrow helper(preorder, left, inorderI - 1)$	1
	$node.right \leftarrow helper(preorder, inorderI + 1, right)$	1
	$return\ node$	1
helper(preorder,left,right) returns the node 9	if $left > right$	1
	$rootVal \leftarrow preorder[preorderI]$	1
	$preorderI++$	1
	new $node(inorderMap[rootVal])$	1
	$inorderI \leftarrow inorderMap[rootVal]$	1
	$node.left \leftarrow helper(preorder, left, inorderI - 1)$	1
	$node.right \leftarrow helper(preorder, inorderI + 1, right)$	1
	$return\ node$	1
8×helper(preorder,left,right) returns nulls	if $left > right$	8
	$return\ null$	8
Total:		103

$T(n = 7) = 13(7) + 12 = 103$✓

Question 12: Diameter of a binary tree

The diameter is the number of edges on the longest path between any two nodes. The path does not have to pass through the root.

- **Input:** The tree from arr
- **Expected output:** 4 (for example the path 21, 12, 5, 8, 17)
- **Twist:** A simple solution calls HEIGHT at every node and takes $\Theta(n^2)$ on a chain. Give a solution that computes the height and the diameter in the same pass, and prove with your frequency count that it is $\Theta(n)$.

Pseudocode

```
Class Solution:
    function diameter(root):
        maxDiameter ← 0
        heightAndDiameter(root, maxDiameter)
        return maxDiameter
    end function
    function heightAndDiameter(node, maxDiameter):
        if node = null {
            return 0
        }
        leftHeight ← heightAndDiameter(node.left, maxDiameter)
        rightHeight ← heightAndDiameter(node.right, maxDiameter)

        maxDiameter ← max(maxDiameter, leftHeight + rightHeight)
        return 1 + max(leftHeight, rightHeight)
    end function
end Class
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	$diameter(root)$	1	1	1
3	$maxDiameter \leftarrow 0$	1	1	1
4	$heightAndDiameter(root, maxDiameter)$	1	1	1
5	$return\ maxDiameter$	1	1	1
8	if $node = null$	1	$2n + 1$	15
9	$return\ 0$	1	$n + 1$	8
11	$leftHeight \leftarrow$ $heightAndDiameter(node.left, maxDiameter)$	1	n	7
12	$rightHeight \leftarrow$ $heightAndDiameter(node.right, maxDiameter)$	1	n	7
14	$maxDiameter \leftarrow$ $max(maxDiameter, leftHeight + rightHeight)$	1	n	7
15	$return\ 1 + max(leftHeight, rightHeight)$	1	n	7

Final complexity

Time complexity: $T(n) = 7n + 6 \rightarrow O(n)$

Explanation: For this algorithm we use a Depth-First traversal, starting from the lowest nodes in the tree for the dimension count. The algorithm starts by initializing a *maxDiameter* variable to store the largest diameter found so far. As the algorithm moves back up other nodes for each node we compare both the accumulated left and right heights up to this node, as well as checking if the sum of these two is greater than the value of *maxDiameter* (this considers the case in which a diameter that doesn't go through the root is larger). Once we reach the root, we simply return whichever value is stored in *maxDiameter* which now holds the value we're looking for.

Twist: A simple solution calls HEIGHT at every node and takes $\Theta(n^2)$ on a chain. Give a solution that computes the height and the diameter in the same pass, and prove with your frequency count that it is $\Theta(n)$.

Worked example

Function Call	Lines executed	Cost
diameter(root) returns the maxDiameter	$diameter(root)$	1
	$maxDiameter \leftarrow 0$	1
	$heightAndDiameter(root, maxDiameter)$	1
	$return\ maxDiameter$	1
helper(preorder,left,right) returns height at node 5 maxDiameter = 4	if $node = null$	1
	$leftHeight \leftarrow heightAndDiameter(node.left, maxDiameter)$	1
	$rightHeight \leftarrow heightAndDiameter(node.right, maxDiameter)$	1
	$maxDiameter \leftarrow max(maxDiameter, leftHeight + rightHeight)$	1
	$return\ 1 + max(leftHeight, rightHeight)$	1
helper(preorder,left,right) returns height at node 12 maxDiameter = 2	if $node = null$	1
	$leftHeight \leftarrow heightAndDiameter(node.left, maxDiameter)$	1
	$rightHeight \leftarrow heightAndDiameter(node.right, maxDiameter)$	1
	$maxDiameter \leftarrow max(maxDiameter, leftHeight + rightHeight)$	1
	$return\ 1 + max(leftHeight, rightHeight)$	1
helper(preorder,left,right) returns height at node 21 maxDiameter = 0	if $node = null$	1
	$leftHeight \leftarrow heightAndDiameter(node.left, maxDiameter)$	1
	$rightHeight \leftarrow heightAndDiameter(node.right, maxDiameter)$	1
	$maxDiameter \leftarrow max(maxDiameter, leftHeight + rightHeight)$	1
	$return\ 1 + max(leftHeight, rightHeight)$	1
helper(preorder,left,right) returns height at node 3 maxDiameter = 0	if $node = null$	1
	$leftHeight \leftarrow heightAndDiameter(node.left, maxDiameter)$	1
	$rightHeight \leftarrow heightAndDiameter(node.right, maxDiameter)$	1
	$maxDiameter \leftarrow max(maxDiameter, leftHeight + rightHeight)$	1
	$return\ 1 + max(leftHeight, rightHeight)$	1
helper(preorder,left,right) returns height at node 8 maxDiameter = 2	if $node = null$	1
	$leftHeight \leftarrow heightAndDiameter(node.left, maxDiameter)$	1
	$rightHeight \leftarrow heightAndDiameter(node.right, maxDiameter)$	1
	$maxDiameter \leftarrow max(maxDiameter, leftHeight + rightHeight)$	1
	$return\ 1 + max(leftHeight, rightHeight)$	1

Function Call	Lines executed	Cost
helper(preorder,left,right) returns height at node 17 maxDiameter = 0	if $node = null$	1
	$leftHeight \leftarrow heightAndDiameter(node.left, maxDiameter)$	1
	$rightHeight \leftarrow heightAndDiameter(node.right, maxDiameter)$	1
	$maxDiameter \leftarrow max(maxDiameter, leftHeight + rightHeight)$	1
	$return\ 1 + max(leftHeight, rightHeight)$	1
helper(preorder,left,right) returns height at node 9 maxDiameter = 0	if $node = null$	1
	$leftHeight \leftarrow heightAndDiameter(node.left, maxDiameter)$	1
	$rightHeight \leftarrow heightAndDiameter(node.right, maxDiameter)$	1
	$maxDiameter \leftarrow max(maxDiameter, leftHeight + rightHeight)$	1
	$return\ 1 + max(leftHeight, rightHeight)$	1
8×helper(preorder,left,right) returns nulls	if $node = null$	8
	$return\ null$	8
Total:		55

$T(n = 7) = 7(7) + 6 = 55$✓

Question 13: Serialize and deserialize a binary tree

Write SERIALIZE, which turns a tree into a list of values using preorder and the marker N for every NULL child, and DESERIALIZE, which rebuilds the identical tree from that list.

- **Input:** The tree from arr
- **Expected output:** 5 12 21 N N 3 N N 8 17 N N 9 N N (DESERIALIZE of this list gives back the tree from arr)
- **Twist:** Why does the list contain exactly 2n + 1 items? Use this fact in the frequency count.

Pseudocode

```
Class Solution:
    function serialize(root):
        s ← []
        serializeHelper(root, s)
        return s
    end function
    function serializeHelper(node, s):
        if node = null {
            s.append("N")
            return
        }

        s.append(string(node.data))
        serializeHelper(node.left, s)
        serializeHelper(node.right, s)
    end function
    index
    n
    function deserialize(nodes)
        index ← 0
        n ← nodes.size()
        root ← deserializeHelper(nodes)
        return root
    end function
    function deserializeHelper(nodes)
        if nodes[index] = "N" {
            index++
            return null
        }
        new node(nodes[index])
        index++

        node.left ← deserializeHelper(nodes)
        node.right ← deserializeHelper(nodes)
        return node
    end function
end Class
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	$serialize(root)$	1	1	1
3	$s \leftarrow []$	1	1	1
4	$serializeHelper(root, s)$	1	1	1
5	$return\ s$	1	1	1
8	if $node = null$	1	$2n + 1$	15
9	$s.append(\text{"}N\text{"})$ 0	1	$n + 1$	8
10	$return$	1	$n + 1$	8
13	$s.append(string(node.data))$	1	n	7
14	$serializeHelper(node.left, s)$	1	n	7
15	$serializeHelper(node.right, s)$	1	n	7
17	$index$	1	1	1
18	n	1	1	1
19	$deserialize(nodes)$	1	1	1
20	$index \leftarrow 0$	1	1	1
21	$n \leftarrow nodes.size()$	1	1	1
22	$root \leftarrow deserializeHelper(nodes)$	1	1	1
23	$return\ root$	1	1	1
26	if $nodes[index] = \text{"}N\text{"}$	1	$2n + 1$	15
27	$index++$	1	$n + 1$	8
28	$return\ null$	1	$n + 1$	8
30	new $node(nodes[index])$	1	n	7
31	$index++$	1	n	7
33	$node.left \leftarrow deserializeHelper(nodes)$	1	n	7
34	$node.right \leftarrow deserializeHelper(nodes)$	1	n	7
35	$return\ node$	1	n	7

Final complexity

Time complexity (Serialize): $T(n) = 7n + 7 \rightarrow O(n)$
Time complexity (Deserialize): $T(n) = 11n + 11 \rightarrow O(n)$

Explanation: This algorithm contains two main functions / sub-algorithms: One for serializing a binary tree, and one for deserializing (reversing) a list of node values. For the serialization, this is completed using a pre-order traversal, appending all visited node values into an empty vector. With this method, as we go down to the nodes of the leaves in the binary tree, we end up adding $n + 1$ null values into this list as "N", which causes the function to iterate a total of $2n + 1$ times. As for the deserializing process, we use a pointer for the index of the list of node values, and once again traverse in pre-order to create each node. In this case for the values "N" in the list we simply return a null pointer, as these are the pointers for the leaves of the tree. Similar to the serializing process, we iterate over a serialized list of nodes, including null nodes, adding up to $2n + 1$ iterations.

To answer the **twist:** These functions process all nodes in a tree, as well as the null children of the leaves of the tree. Considering we have n nodes, the number of null children is given as $n + 1$, giving a total frequency count for some operations of $2n + 1$.

Worked example - Serialize

Function Call	Lines executed	Cost
diameter(root) returns s	$serialize(root)$	1
	$s \leftarrow []$	1
	$serializeHelper(root, s)$	1
	$return\ s$	1
serializeHelper(node, s) Node: 5	if $node = null$	1
	$s.append(string(node.data))$	1
	$serializeHelper(node.left, s)$	1
	$serializeHelper(node.right, s)$	1
serializeHelper(node, s) Node: 12	if $node = null$	1
	$s.append(string(node.data))$	1
	$serializeHelper(node.left, s)$	1
	$serializeHelper(node.right, s)$	1
serializeHelper(node, s) Node: 21	if $node = null$	1
	$s.append(string(node.data))$	1
	$serializeHelper(node.left, s)$	1
	$serializeHelper(node.right, s)$	1
2×serializeHelper(node, s) Node: null	if $node = null$	2
	$s.append("N")$; $return$	4
serializeHelper(node, s) Node: 3	if $node = null$	1
	$s.append(string(node.data))$	1
	$serializeHelper(node.left, s)$	1
	$serializeHelper(node.right, s)$	1
2×serializeHelper(node, s) Node: null	if $node = null$	2
	$s.append("N")$; $return$	4
serializeHelper(node, s) Node: 8	if $node = null$	1
	$s.append(string(node.data))$	1
	$serializeHelper(node.left, s)$	1
	$serializeHelper(node.right, s)$	1
serializeHelper(node, s) Node: 17	if $node = null$	1
	$s.append(string(node.data))$	1
	$serializeHelper(node.left, s)$	1
	$serializeHelper(node.right, s)$	1
2×serializeHelper(node, s) Node: null	if $node = null$	2
	$s.append("N")$; $return$	4
serializeHelper(node, s) Node: 9	if $node = null$	1
	$s.append(string(node.data))$	1
	$serializeHelper(node.left, s)$	1
	$serializeHelper(node.right, s)$	1
2×serializeHelper(node, s) Node: null	if $node = null$	2
	$s.append("N")$; $return$	4
Total:		56

$T(n = 7) = 7(7) + 7 = 56$✓

Worked example - Serialize

Function Call	Lines executed	Cost
diameter(root) returns the root	$index$;	1
	n	1
	$deserialize(nodes)$	1
	$index \leftarrow 0$	1
	$n \leftarrow nodes.size()$	1
	$root \leftarrow deserializeHelper(nodes)$	1
	$return\ root$	1
deserializeHelper(nodes) Index: 0 Node: 5	if $nodes[index] =$ "N"	1
	$newnode(nodes[index])$	1
	$index++$	1
	$node.left \leftarrow deserializeHelper(nodes)$	1
	$node.right \leftarrow deserializeHelper(nodes)$	1
	$return\ node$	1
deserializeHelper(nodes) Index: 1 Node: 12	if $nodes[index] =$ "N"	1
	$newnode(nodes[index])$	1
	$index++$	1
	$node.left \leftarrow deserializeHelper(nodes)$	1
	$node.right \leftarrow deserializeHelper(nodes)$	1
	$return\ node$	1
deserializeHelper(nodes) Index: 2 Node: 21	if $nodes[index] =$ "N"	1
	$newnode(nodes[index])$	1
	$index++$	1
	$node.left \leftarrow deserializeHelper(nodes)$	1
	$node.right \leftarrow deserializeHelper(nodes)$	1
	$return\ node$	1
2×deserializeHelper(nodes) Index: 3, 4 Node: N, N	if $nodes[index] =$ "N"	2
	$index++$	2
	$return\ null$	2
deserializeHelper(nodes) Index: 5 Node: 3	if $nodes[index] =$ "N"	1
	$newnode(nodes[index])$	1
	$index++$	1
	$node.left \leftarrow deserializeHelper(nodes)$	1
	$node.right \leftarrow deserializeHelper(nodes)$	1
	$return\ node$	1
2×deserializeHelper(nodes) Index: 6, 7 Node: N, N	if $nodes[index] =$ "N"	2
	$index++$	2
	$return\ null$	2
deserializeHelper(nodes) Index: 8 Node: 8	if $nodes[index] =$ "N"	1
	$newnode(nodes[index])$	1
	$index++$	1
	$node.left \leftarrow deserializeHelper(nodes)$	1
	$node.right \leftarrow deserializeHelper(nodes)$	1
	$return\ node$	1

Function Call	Lines executed	Cost
deserializeHelper(nodes) Index: 9 Node: 17	if $nodes[index] = "N"$	1
	$newnode(nodes[index])$	1
	$index++$	1
	$node.left \leftarrow deserializeHelper(nodes)$	1
	$node.right \leftarrow deserializeHelper(nodes)$	1
	$return\ node$	1
2×deserializeHelper(nodes) Index: 10, 11 Node: N, N	if $nodes[index] = "N"$	2
	$index++$	2
	$return\ null$	2
deserializeHelper(nodes) Index: 12 Node: 9	if $nodes[index] = "N"$	1
	$newnode(nodes[index])$	1
	$index++$	1
	$node.left \leftarrow deserializeHelper(nodes)$	1
	$node.right \leftarrow deserializeHelper(nodes)$	1
	$return\ node$	1
2×deserializeHelper(nodes) Index: 13, 14 Node: N, N	if $nodes[index] = "N"$	2
	$index++$	2
	$return\ null$	2
Total:		73

$T(n = 7) = 9(7) + 10 = 73$✓

Question 14: Is the tree a valid binary search tree

Return true if every node is larger than all values in its left subtree and smaller than all values in its right subtree. Hint: checking only a node against its two children is not enough; pass down an allowed range (low, high).

- **Input:** Tree P from arr; tree S given in level order: 9, 5, 17, 3, 8, 12, 21.
- **Expected output:** ISBST(P) = false; ISBST(S) = true (its inorder is 3, 5, 8, 9, 12, 17, 21)
- **Twist:** Give a second solution that uses an inorder traversal. Compare both frequency counts. What is the best case for each?

Pseudocode

```
function ISBST(root, min, max):
    if node = null {
        return true
    }
    if root.data ≤ min or root.data ≥ max {
        return false
    }
    l ← ISBST(root.left, min, root.data)
    r ← ISBST(root.right, root.data, max)
    return l and r
end function
```

Frequency count

Line	Operation	Cost	Frequency	Count for example
2	if $node = null$	1	$2n + 1$	15
3	$return\ true$	1	$n + 1$	8
5	if $root.data \leq min$ or $root.data \geq max$	1	n	7
6	$return\ false$	1	f	0
8	$l \leftarrow ISBST(root.left, min, root.data)$	1	n	14
9	$r \leftarrow ISBST(root.right, root.data, max)$	1	n	14
10	$return\ l$ and r	1	$n - f$	7

Final complexity

Time complexity: $T(n) = 7n + 2 \rightarrow O(n)$

Explanation: This algorithm uses the same logic as in previous problems: Recursively go down to the lower levels of the tree, and as it is being built up checking that every parent - child pair fulfills the rule. If a node is found in which the rule is broken, it returns a value of $false$ which returns all the way up to the original function call.

Worked example - S

Function Call	Lines executed	Cost
ISBST(9) returns true	if $node = null$	1
	if $root.data \leq min$ or $root.data \geq max$	1
	$l \leftarrow ISBST(root.left, min, root.data)$	1
	$r \leftarrow ISBST(root.right, root.data, max)$	1
	return l and r	1
ISBST(5) returns true	if $node = null$	1
	if $root.data \leq min$ or $root.data \geq max$	1
	$l \leftarrow ISBST(root.left, min, root.data)$	1
	$r \leftarrow ISBST(root.right, root.data, max)$	1
	return l and r	1
ISBST(17) returns true	if $node = null$	1
	if $root.data \leq min$ or $root.data \geq max$	1
	$l \leftarrow ISBST(root.left, min, root.data)$	1
	$r \leftarrow ISBST(root.right, root.data, max)$	1
	return l and r	1
ISBST(3) returns true	if $node = null$	1
	if $root.data \leq min$ or $root.data \geq max$	1
	$l \leftarrow ISBST(root.left, min, root.data)$	1
	$r \leftarrow ISBST(root.right, root.data, max)$	1
	return l and r	1
ISBST(8) returns true	if $node = null$	1
	if $root.data \leq min$ or $root.data \geq max$	1
	$l \leftarrow ISBST(root.left, min, root.data)$	1
	$r \leftarrow ISBST(root.right, root.data, max)$	1
	return l and r	1
ISBST(12) returns true	if $node = null$	1
	if $root.data \leq min$ or $root.data \geq max$	1
	$l \leftarrow ISBST(root.left, min, root.data)$	1
	$r \leftarrow ISBST(root.right, root.data, max)$	1
	return l and r	1
ISBST(21) returns true	if $node = null$	1
	if $root.data \leq min$ or $root.data \geq max$	1
	$l \leftarrow ISBST(root.left, min, root.data)$	1
	$r \leftarrow ISBST(root.right, root.data, max)$	1
	return l and r	1
8*maxDepth(NULL) returns true	if $node = null$	8
	$return\ true$	8
Total:		51

$T(n = 7) = 7(7) + 2 = 51$✓

Twist: Give a second solution that uses an inorder traversal. Compare both frequency counts. What is the best case for each?

Twist - Pseudocode

```
1  Class Solution:
2      prev ← null
3      function ISBST(root):
4          if node ← null {
5              return true
6          }
7          if not ISBST(root.left) {
8              return false
9          }
10         if prev ≠ null and root.data ≤ prev.data) {
11             return false
12         }
13         prev ← root
14         r ← ISBST(root.right)
15         return r
16     end function
17 end Class
```

Twist - Frequency count

Line	Operation	Cost	Frequency	Count for example
2	$prev \leftarrow null$	1	1	1
3	$ISBST(root)$	1	1	1
4	if $node = null$	1	$2n + 1$	15
5	$return\ true$	1	$n + 1$	8
7	if not $ISBST(root.left)$	1	n	3
8	$return\ false$	1	h_f	-
10	if $prev \neq null$ and $root.data \leq prev.data$	1	n	7
11	$return\ false$	1	-	-
13	$prev \leftarrow root$	1	n	1
14	$r \leftarrow ISBST(root.right)$	1	n	1
15	$return\ r$	1	n	7

Twist - Final complexity

Explanation: With this algorithm, we check the nodes of the tree using in-order traversal, which places the nodes in order from far left to far right. With this order, we simply have to check that every sequential pair of node values follows the rule where the previous value is less than the next. If any node breaks this rule, it returns a value of $false$ which causes the function to break out and return $false$ without moving on to the next nodes. Due to this, the best case scenario for this algorithm occurs when the two left-most nodes don't follow the rule, as the algorithm won't move to any other node in the tree.